

Highlights

Hierarchical role-based access control on Hyperledger Fabric for agricultural edge IoT: a 61-day field case study

Forcha Glen Beloa, Michael Ekonde Sone, Elie Fute Tagne, André Paul Mvogo Asse

- A 61-day field deployment of HRBAC on Hyperledger Fabric with 50 sensors across 4 zones, recording 209 000 authorization decisions.
- Policy-evaluation transactions cost 280 ms against 1 220 ms for sensor writes; day-level differences among 7 operation types are equivalent within ± 10 ms.
- Access control reduced throughput 9.9% across a tenfold concurrency range; both arms saturated at the same offered load.
- Endorsing peers cut write latency 32.7% from 4 to 32 without raising throughput.
- 29.5% of 200 revocation lifecycle stages were delayed by connectivity, a behaviour short benchmarks cannot observe.

Hierarchical role-based access control on Hyperledger Fabric for agricultural edge IoT: a 61-day field case study^{*}

Forcha Glen Beloa^{a,*}, Michael Ekonde Sone^a, Elie Fute Tagne^a and André Paul Mvogo Asse^b

^aDepartment of Computer Engineering, Faculty of Engineering and Technology, University of Buea, Buea, Cameroon

^bÉcole Nationale Supérieure Polytechnique de Douala, Université de Douala, Douala, Cameroon

ARTICLE INFO

Keywords:

Hierarchical role-based access control
Hyperledger Fabric
Agricultural IoT
Edge computing
Performance measurement
Certificate revocation
Reproducible research

ABSTRACT

Blockchain access control for the Internet of Things is usually evaluated on short laboratory runs reporting a peak transaction rate and a mean latency. Field evidence about how such a system behaves over months, and about which part of the request path the time is spent in, is scarce. We report a 61-day deployment of a hierarchical role-based access control (HRBAC) framework on Hyperledger Fabric 2.5, running on a 2 ha farm in Sfax, Tunisia with 50 sensors, 4 Raspberry Pi 4B gateways and 4 zones. The framework provides seven roles with zone scoping enforced from X.509 extensions, temporal validity on every assignment, and a cascading enrollment protocol through which ESP32-class sensors obtain certificates via their gateway without running a TLS handshake. Over 209 000 recorded authorization decisions, policy evaluation transactions took 280 ms on average while sensor-write transactions took 1 220 ms, a difference of 940 ms between the two operation classes. Using day-level means as the experimental unit, latency differences among 7 policy-evaluation operation types were statistically equivalent within a prespecified ± 10 ms margin (TOST $p < 0.001$). Access control reduced throughput by 9.9% on average across a tenfold concurrency range, with no detected trend against load, and the configurations with and without it saturated at the same offered load, placing the limiting resource downstream of the policy layer. Increasing endorsing peers from 4 to 32 reduced write latency 32.7% while throughput remained flat. Revocation publication took 60.4 s and gossip 4.21 min, but 29.5% of 200 recorded lifecycle stages were marked delayed by connectivity. All 8 000 scripted authorization-boundary cases were rejected. We report the measurement limits candidly, including which claims the trace cannot support, and release the data and analysis pipeline in full.

1. Introduction

A precision farm is a multi-stakeholder system before it is a technical one. Soil probes, weather stations and irrigation actuators produce readings continuously [54], and the people who consume those readings want different and partly conflicting things from them. Farmers need operational control. Agronomists need history but no actuation rights. Certification bodies auditing organic or GlobalGAP compliance need evidence of practice without visibility into the agronomic detail a farm treats as commercially sensitive. Supply-chain partners need provenance but not real-time operations. Regulators need environmental records.

Handing this authorization matrix to a single server concentrates the risk in one place. An attacker who takes that server gains not only the data but, on a farm with networked valves, the irrigation itself [7], and the audit log that would reconstruct the incident is written by the machine that was compromised. Surveying agricultural IoT deployments, Ferrag et al. [12] found most lacked fine-grained access control at all. Kamilaris et al. [20] showed that blockchain work in agriculture had settled almost entirely on post-harvest supply-chain tracking, and recent surveys of IoT, AI and blockchain convergence in agriculture reach the same

^{*} The trace dataset, chaincode, firmware, formal specification and the analysis pipeline that regenerates every number, table and figure in this article are available at <https://github.com/forchag/blockchain-fet>.

^{*} Corresponding author

 glenbeloa@gmail.com (F.G. Beloa); ekonde.sone@gmail.com (M.E. Sone); eliefute@yahoo.fr (E.F. Tagne); mvogoasseandre@gmail.com (A.P.M. Asse)

ORCID(s): 0009-0003-1972-6100 (F.G. Beloa)

conclusion about pre-harvest authorization [35]. Putting role assignments, revocations and access decisions on a permissioned ledger removes the single point of trust. That much is settled.

What is not settled is the cost, and specifically where the cost falls.

1.1. The measurement gap

Among the systems surveyed in Table 1, evaluation is typically a laboratory or simulated run of hours to days, reported as a peak throughput and a mean latency. Both are aggregates over a pipeline with at least four separable stages: identity retrieval, policy evaluation, consensus ordering, and commitment with world-state update. Told that a transaction takes 1 220 ms, an engineer cannot tell whether to rewrite the policy engine, restructure the endorsement policy, replace the ordering service, or accept the figure as a floor imposed by consensus.

Separating those stages properly requires timestamping one transaction at each hand-off inside Fabric. Our instrumentation does not do that. It records end-to-end latency per transaction, which supports comparison between operation classes but not layer-by-layer attribution. We are explicit throughout about which of the two a given number is, and Section 7.4 lists the instrumentation a genuine stage decomposition would require.

Abang et al. [1], reviewing latency modelling for Hyperledger Fabric from an IoT perspective, make a closely related point. The literature models latency in the abstract but rarely connects those models to the behaviour of a specific deployed use case. They decompose Fabric latency into endorsement, ordering and validation phases and identify block size, endorsement policy, peer count and state database as the parameters that govern it. What has been missing is a deployment that measures those phases separately, under a real workload, for long enough that the slow-moving parts of the system have a chance to misbehave.

The intuitive expectation turns out to be wrong, which is the main reason we think the measurement is worth reporting. Access control is the component under study, so it is natural to assume it dominates the cost. It does not. The authorization decision is small and remarkably stable, and the expensive work happens in the layer that access-control papers usually treat as background infrastructure.

A second gap is temporal. Certificate lifecycle management, revocation propagation and enrollment recovery are the mechanisms most likely to fail in the field and least likely to be exercised by an afternoon of benchmarking. A revocation pipeline that behaves perfectly across fifty laboratory trials can still show a materially different delay distribution across two months of intermittent rural connectivity. Reporting only the happy path understates operational risk in a way that matters to anyone deciding whether to deploy.

1.2. Contributions

We built an HRBAC framework on Hyperledger Fabric 2.5 for the constraints of agricultural edge deployment, ran it on a working farm for 61 days, and instrumented the authorization and end-to-end transaction paths.

1. **A longitudinal field record.** 61 days of continuous operation on a working farm, 209 000 authorization decisions and 146 400 sensor writes, covering behaviour that hours-long testbed runs cannot reach: connectivity outages, enrollment recovery, seasonal role expiry and revocation under intermittent links (Sections 6 and 7.3).
2. **An operation-class latency comparison with a stated equivalence margin.** Policy-evaluation transactions cost 280 ms and sensor writes 1 220 ms. Using day-level means rather than individual transactions as the experimental unit, differences among 7 policy-evaluation operation types are equivalent within a prespecified ± 10 ms margin (Section 6.1). We also set out why this should not be read as a statement about privilege level.
3. **A saturation result that places the limit below the policy layer.** Access control reduced throughput by 9.9% with no detected trend across a tenfold concurrency range, and both arms saturated at the same offered load, so the limiting resource lies downstream of authorization (Section 6.2). We do not claim to have identified which downstream resource it is.
4. **A latency-capacity separation for peer provisioning.** An eightfold increase in endorsing peers cut write latency 32.7% while throughput stayed flat (Section 6.3).

5. **An artifact with its limits documented.** Traces with checksums, implementation, formal specification and an analysis pipeline that regenerates every number, table and figure, together with an explicit account of the claims the data cannot support (Sections 5.4 and 7.4).

Section 2 positions the work. Section 3 presents the access-control model and the enrollment protocol, Section 4 the implementation, Section 5 the measurement methodology, Section 6 the results, Section 7 what follows from them for system design, and Section 8 concludes.

2. Related work

2.1. Access control for constrained devices

Role-based access control as formalised by Sandhu et al. [38] and standardised by NIST into Core, Hierarchical and Separation-of-Duty variants [13] is still the reference model for multi-stakeholder authorization. Its IoT adaptations share a structural assumption that becomes awkward in the field: some trusted decision component has to be reachable at the moment access is requested. RBAC extensions for IoT [51], attribute-based evaluation [17, 40, 39] and capability-token delegation [16] place that trust in a policy engine, an attribute authority and a credential issuer respectively. Sciancalepore et al. [39] quantify what open-standard authorization stacks cost on constrained hardware, which is the empirical case for moving policy evaluation off the sensor entirely.

Each of these reduces part of the problem and keeps a central dependency somewhere. Ye et al. [51] and Servos and Osborn [40] retain a central enforcement point. Hernández-Ramos et al. [16] move authorization into tokens, which distributes revocation instead of solving it. Alshehri and Sandhu [4] split the model, giving RBAC to humans and capability-based control to devices, and the role store stays centralised. Novo [29] argued early that the role store itself belongs on a distributed ledger. We take that as given and ask what it costs.

2.2. Blockchain-based access control and how it is evaluated

Capability-token systems on public chains established feasibility without practicality. FairAccess [31] encodes grants as Bitcoin-like transfers and inherits confirmation windows measured in minutes. BlendCAC [50] uses ERC-721 tokens and incurs per-transaction fees that no sensor duty cycle can absorb. Neither addresses role hierarchy, geographic scoping or seasonal expiry, which is what an agricultural deployment actually needs.

Permissioned platforms remove the latency and fee obstacles, and Fabric-based IoT work has grown accordingly. Shih et al. [41] showed Fabric access control running on Raspberry Pi 4B ARM64 hardware. Pajooh et al. [32] and Ke and Park [21] contributed testbed architectures and performance models. Putra et al. [33] added trust and reputation. Usman et al. [46] enforce domain policies by smart contract on cloud-hosted nodes, where memory and thermal limits do not apply. González et al. [15] report a Fabric-based IoT data marketplace at scale and, like most of this literature, evaluate it with aggregate throughput and latency from a controlled setting. Idrissi and Palmieri [19] give an end-to-end Fabric latency breakdown on constrained ARM hardware in a healthcare context, and their decomposition approach is the closest methodological precedent for ours. Tzenetopoulos et al. [45] established a reference point for Fabric on Raspberry Pi 3B+ hardware that we compare against in Section 7.2.

Table 1 summarises the evaluation methodology of these systems. The pattern is consistent. Results come from laboratory testbeds or simulation, run for hours or days, and are reported as aggregates. We have not found prior work that separates the authorization decision from consensus and commitment cost on the same deployment, or that reports revocation behaviour over a multi-week window. That combination, rather than any single mechanism in Section 3, is the contribution.

2.3. Agricultural IoT and edge infrastructure

Blockchain in agriculture has concentrated on traceability after harvest [44, 24, 6], where the unit of interest is a shipment rather than a zone-scoped actuation. Safeer et al. [36] combine computer vision, IoT and blockchain for climate-smart cultivation and, like most pre-harvest work, treat authorization as an implementation detail rather than a measured component. Liu et al. [25] and Wang et al. [47] reduce processing latency at the agricultural edge without addressing the authorization layer above it. Mohanta et

Table 1

Evaluation methodology in representative blockchain access-control systems. *Decomposed* indicates whether the authorization decision cost is separated from consensus and commitment cost. N/R, not reported.

System	Platform	Setting	Duration	Decomposed	Revocation measured
FairAccess [31]	Bitcoin	Analytical	N/A	No	No
BlendCAC [50]	Ethereum	Testbed	Hours	No	No
Novo [29]	Ethereum	Testbed	Hours	No	No
Shih et al. [41]	Fabric	Lab (ARM64)	Days	No	No
Zaidi et al. [53]	Fabric	Simulation	N/A	No	No
Pajooh et al. [32]	Fabric	Lab	N/R	No	No
Putra et al. [33]	Fabric	Lab	N/R	No	No
Usman et al. [46]	Fabric	Cloud	Days	No	No
González et al. [15]	Fabric	Lab	N/R	No	No
Idrissi & Palmieri [19]	Fabric	Lab (ARM)	N/R	Partial	No
Bandara et al. [6]	Fabric	Simulation	N/A	No	No
This work	Fabric	Field	61 days	Yes	Yes

al. [26] apply smart-contract enforcement to smart-grid security, a neighbouring domain with comparable device constraints and a similarly centralised starting point.

On identity infrastructure, W3C Decentralized Identifiers [48] and gateway-assisted verification for constrained devices [11, 42] offer an eventual migration away from X.509, which we treat as future work rather than a deployed alternative. On the radio side, Adelantado et al. [2] characterise the LoRaWAN capacity limits that bound achievable sensor density per gateway, a constraint that becomes visible in Section 6.6.

3. Access-control design

3.1. Deployment structure and threat model

The deployment has three tiers, shown in Figure 1. ESP32-WROOM-32 sensor nodes (dual-core Xtensa LX6 at 240 MHz, 520 kB SRAM) sample soil and environmental data, sign each reading with an Ed25519 key held in one-time-programmable eFuse, and transmit over LoRa. Raspberry Pi 4B zone gateways (quad-core Cortex-A72, 4 GB LPDDR4, TPM 2.0) receive those payloads, verify signatures, endorse Fabric transactions and cache policy decisions locally. A Fabric network of four peers, one per zone, three Raft orderers and CouchDB 3.2 for world state maintains the ledger. Gateways interconnect over a BATMAN-adv mesh [27] that reroutes around individual gateway failures and carries Fabric gossip and certificate revocation list (CRL) traffic between zones.

Following the permissioned-blockchain model in which every participant is credentialed [5], orderers and the certificate authority are operated by vetted institutional entities and treated as non-malicious. Raft gives crash-fault tolerance with $2f + 1$ nodes [30], so the three-node cluster survives one orderer failure but not a reachable orderer that behaves maliciously. That is a deliberate boundary and it is also a throughput boundary. Byzantine-tolerant ordering such as HotStuff [52], PBFT [8] or the SmartBFT service in Fabric v3 [18] would raise the consensus cost while widening the threat model, a trade-off the measurements in Section 6.2 let us quantify rather than guess at.

Sensors, client devices and every wireless channel are treated as potentially compromised. Physical tampering and side-channel extraction against cryptographic accelerators sit outside the evaluated scope and are revisited in Section 7.5.

3.2. Role hierarchy

Following the treatment of hierarchical role administration in ARBAC97 [37], a partial order $\preceq$ over the role set R defines inheritance, with $r_i \preceq r_j$ iff $\text{perm}(r_i) \subseteq \text{perm}(r_j)$. The deployed hierarchy (Figure 2) is a human chain and a separate device chain, both below Admin:

$$\text{SupplyChain} \preceq \text{Certifier} \preceq \text{Agronomist} \preceq \text{Farmer} \preceq \text{Admin} \quad (1)$$

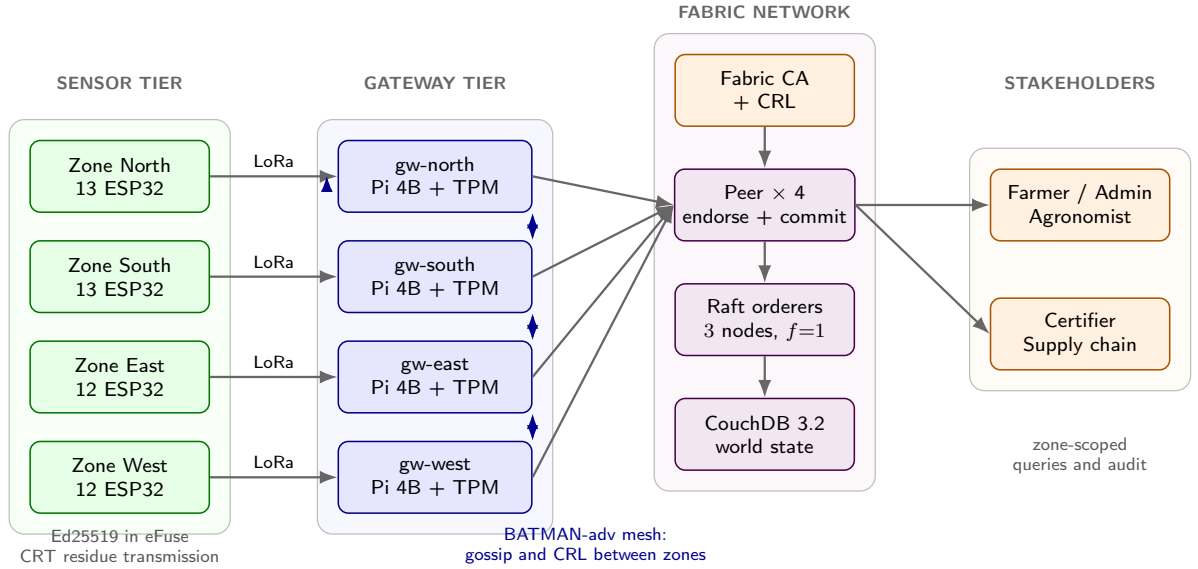


Figure 1: Deployment architecture. Sensors reach the ledger only through their zone gateway, which is also their registration authority during enrollment. Zone membership is carried in the device certificate and re-checked by chaincode on every transaction.

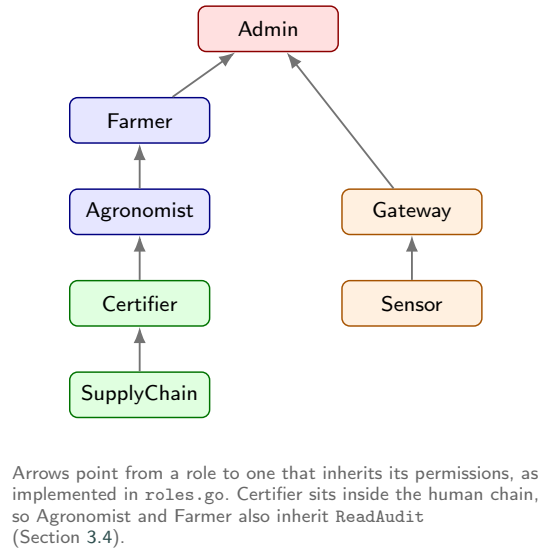


Figure 2: Seven-role hierarchy as deployed. Certifier lies on the human inheritance chain rather than beside it, so audit-log access is inherited upward by Agronomist and Farmer.

$$\text{Sensor} \preceq \text{Gateway} \preceq \text{Admin} \quad (2)$$

Placing Certifier inside the human chain rather than beside it has a consequence we did not intend when the model was written and did not notice until the implementation was audited against the manuscript for this article: Agronomist and Farmer inherit Certifier's permissions, including `ReadAudit`. The deployment therefore did *not* confine audit-log access to auditors, and the separation this article's introduction motivates was weaker in the running system than in the design that motivated it. Section 3.4 records the full discrepancy. We report the model as deployed, because that is the model the measurements describe.

Table 2

Effective permissions per role, generated from the deployed chaincode (`roles.go`) rather than transcribed. Inheritance along the human chain of Equation (1) is why *Read audit* reaches Agronomist and Farmer.

Permission	Sensor	Gateway	Farmer	Agronomist	Certifier	Supply	Admin
Read own	–	–	✓	✓	✓	✓	✓
Read zone	–	✓	✓	✓	✓	–	✓
Read all	–	–	–	–	–	–	✓
Write sensor	✓	✓	✓	–	–	–	✓
Control zone	–	✓	✓	✓	–	–	✓
Control all	–	–	–	–	–	–	✓
Read audit	–	–	✓	✓	✓	–	✓
Issue CZT	–	–	✓	✓	–	–	✓
Manage roles	–	–	–	–	–	–	✓
Admin all	–	–	–	–	–	–	✓

Separating auditors from operational staff is the requirement that motivated the model: an auditor should read compliance evidence without inheriting the agronomic permissions a farm treats as proprietary. Section 3.4 explains why the deployed hierarchy does not achieve it. The effective permission set of user u in a session is

$$\text{eff}(u) = \bigcup_{r \in \text{active}(u)} \bigcup_{r' \preceq r} pa(r'), \quad (3)$$

where $pa(r)$ are the permissions assigned directly to r and $\text{active}(u)$ the non-expired roles held by u . An access request (u, op, res, t) is granted iff

$$\begin{aligned} &\exists r \in \text{active}(u, t) : (op, res) \in \text{eff}(r) \\ &\wedge (\text{zone}(u) = \text{zone}(res) \vee \text{czt}(u, \text{zone}(res), t)), \end{aligned} \quad (4)$$

where $\text{czt}(u, z, t)$ holds iff u has a valid administrator-issued cross-zone token for z at time t , and $\text{active}(u, t) = \{r \mid \exists \text{assign}(u, r, t_s, t_e) : t_s \leq t \leq t_e\}$. Table 2 gives the permission matrix that results.

3.3. Zone and temporal scoping

Geographic farm regions (North, South, East, West) define the zone model, and it is enforced at three independent points rather than one. Each device certificate carries its zone in an X.509 Subject Alternative Name extension under a private OID arc. Each gateway accepts certificate signing requests only from sensors declaring its own zone. At the chaincode level every zone-scoped transaction calls `validateZoneAccess`, which parses the caller's zone from the certificate DER bytes by ASN.1 decoding rather than by matching against the PEM text. That last detail matters: string matching against a common name is spoofable by an attacker who controls the subject field, and ASN.1 decoding of the extension is not.

Cross-zone tokens are administrator-issued and recorded immutably with subject, permitted zones, validity window and stated purpose. An agronomist auditing a neighbouring zone during a multi-farm review gets a bounded token instead of a reissued certificate, so the grant expires on its own and leaves a record of why it was made.

Temporal constraints run in two independent layers. The chaincode compares an `ExpiresAt` field on the role assignment against the Fabric transaction timestamp before every decision, and the Fabric MSP independently validates the X.509 `NotAfter` field on every incoming transaction. A seasonal worker granted the Farmer role for a harvest window therefore loses access at expiry without an administrator doing anything, and the expiry is itself recorded as a new immutable audit entry referencing the original assignment. Gateways carry GPS modules whose one-pulse-per-second output gives a sub-microsecond local time reference, distributed onward by NTP at millisecond precision, comfortably inside the ± 5 -minute enforcement window.

Algorithm 1 Access decision (**CheckAccess**)**Input:** request (u, op, res, t) , caller certificate C **Output:** GRANT or DENY. Every path writes an audit entry, so every invocation mutates state and commits through ordering.

```

1:  $n \leftarrow \text{NONCE}(req)$ 
2: if SEENBEFORE( $n$ ) then
3:   AUDIT(DENY, REPLAYDETECTED); return DENY
4: end if
5:  $z_u \leftarrow \text{PARSEZONEASN1}(C)$  ▷ DER bytes, not PEM text
6: if ONCRL( $C$ ) then
7:   AUDIT(DENY, CERTREVOKED); return DENY
8: end if
9:  $A \leftarrow \text{ROLEASSIGNMENTS}(u)$ 
10:  $R_{\text{act}} \leftarrow \emptyset$ 
11: for  $a \in A$  do
12:   if  $a.\text{ExpiresAt} < t$  then
13:     REVOKEROLE( $a$ ) ▷ state-mutating: forces endorsement
14:   else
15:      $R_{\text{act}} \leftarrow R_{\text{act}} \cup \{a.\text{role}\}$ 
16:   end if
17: end for
18: if  $R_{\text{act}} = \emptyset$  then
19:   AUDIT(DENY, ROLEINSUFFICIENT); return DENY
20: end if
21:  $P \leftarrow \bigcup_{r \in R_{\text{act}}} \bigcup_{r' \preceq r} pa(r')$  ▷ inheritance closure, depth  $\leq 3$ 
22: if  $(op, res) \notin P$  then
23:   AUDIT(DENY, OPNOTPERMITTED); return DENY
24: end if
25:  $z_r \leftarrow \text{ZONEOF}(res)$ 
26: if  $z_u \neq z_r \wedge \neg \text{VALIDCZT}(u, z_r, t)$  then
27:   AUDIT(DENY, ZONEMISMATCH); return DENY
28: end if
29: RECORDNONCE( $n$ ); AUDIT(GRANT,  $(op, res, z_r)$ )
30: return GRANT

```

Algorithm 1 states the decision procedure that Equation (4) describes. Two properties of it matter for the measurements in Section 6.1.

The work is bounded and structurally independent of which operation was requested: the hierarchy is walked to a fixed depth of at most three, and the zone check is a single comparison against a parsed certificate field. That is a property of the code, and it is the mechanism we would expect to produce similar costs across operations.

Every path through the algorithm writes an audit entry, including every denial. A **CheckAccess** invocation is therefore state-mutating and commits through ordering exactly as a sensor write does. We stress this because it is easy to assume that a permission check is a read: it is not, in this design. The consequence is that Section 6.1 compares two classes of committed transaction, and neither class avoids consensus. Expired assignments are additionally revoked inside the same transaction that denies the request, which keeps the audit trail single-sourced at the cost of making even a read-intent check a write.

3.4. Where the implemented model differs from the design

Preparing this article we audited the deployed chaincode against the model as written, and found the two had diverged. Since the measurements describe the deployed system, the model presented above is the deployed one, and this subsection records what the divergence was. We think reporting it is more useful than quietly re-aligning the text, because the failure mode is a common one: a policy that reads correctly in a diagram and inherits differently in code.

Three differences matter.

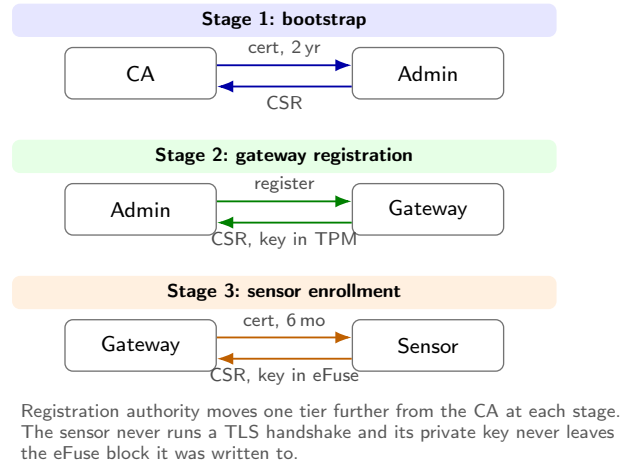


Figure 3: Three-stage cascading enrollment.

Audit access is inherited. Certifier sits on the human chain, so Agronomist and Farmer receive **ReadAudit** through inheritance. The intended model made Certifier a sibling of the operational roles precisely to prevent this. Nothing in the deployment enforced the separation the introduction motivates, and the 61 days of operation therefore do not evidence it.

Certifier holds ReadZone. An auditor could read zone sensor data directly. The intended model gave certifiers derived compliance records only. Implementing that distinction needs a resource discriminator that separates raw sensor state from compliance views, which the chaincode does not have: **CheckAccess** matches on permission and zone alone.

Operational permissions differ. Agronomist holds **ControlZone** in the deployment, while the intended model gave agronomists read access without actuation. Farmer holds neither **ReadAll** nor **ControlAll**.

None of this affects the latency, throughput or revocation measurements, which do not depend on which permissions a role holds. It does bear on Section 6.5: the scripted campaign tests the boundaries the deployed policy defines, so it confirms that the implemented model was enforced, not that the intended model was. Closing the gap means either adding the resource discriminator and moving Certifier off the human chain, or restating the design to match the code. We consider the former the correct repair and have not made it here, because changing the policy would invalidate the deployment these measurements describe.

3.5. Cascading enrollment

An ESP32 cannot hold the TLS handshake state that direct interaction with a certificate authority requires. Measurements of lightweight secure IoT stacks show handshake buffers dominating constrained-device memory budgets [34], and the mbedTLS handshake path on ESP-IDF consumes roughly 70 kB of heap, which leaves too little of the 520 kB SRAM budget for the application that is supposed to run alongside it. The protocol therefore pushes registration authority down the tier structure in three stages (Figure 3). Stage 1 issues long-lived administrator credentials. Stage 2 registers each gateway, which stores its private key in TPM 2.0. In Stage 3 the gateway acts as a local registration authority, detailed in Algorithm 2.

Because the private key is generated on the sensor and never transmitted, compromising a gateway does not yield sensor signing keys. Re-provisioning a node whose key is lost needs physical access, since eFuse programming is irreversible.

The key is *not* read-protected in the deployed firmware, and we want to be precise about why. Signing uses wolfSSL in software, so **private_key_read** copies the 32 bytes out of eFuse block 3 into RAM for each signature and zeroes the buffer afterwards. Setting the eFuse read-disable bit would make that copy fail and break signing outright: the ESP32 used here has no hardware Ed25519 engine and no key-wrapping peripheral that could sign on the key's behalf. Read protection is therefore a hardening step that a different part is needed to support, for instance the HMAC and digital-signature peripherals on the ESP32-S3, rather than something the deployed nodes achieve. The eFuse gives durability and resistance to reflashing, not extraction

Algorithm 2 Stage 3: sensor enrollment through the zone gateway**Input:** sensor s with declared zone z_s , gateway g with zone z_g **Output:** zone-bound certificate in RTC memory, private key sealed in eFuse**On the sensor:**1: $(sk, pk) \leftarrow \text{ED25519KEYGEN}()$ 2: $\text{EFUSEWRITE}(\text{BLK3}, sk)$

▷ one-time programmable, survives reflash

3: $csr \leftarrow \text{MAKECSR}(pk, z_s); \sigma \leftarrow \text{SIGN}_{sk}(csr)$ 4: send (csr, σ) to g over LoRa**On the gateway:**5: **if** $z_s \neq z_g$ **then**6: **return** reject

▷ zone binding enforced at the RA

7: **end if**8: **if** $\neg \text{RATELIMITOK}(src)$ **then**9: **return** reject10: **end if**11: **if** $\neg \text{VERIFYSIG}(csr, \sigma, pk)$ **then**12: **return** reject13: **end if**14: $cert \leftarrow \text{CAISSUE}(csr, \text{SAN}=z_g, \text{NOTAFTER}=t+6 \text{ months})$ 15: **return** $cert$ to s **On the sensor, each wake:**16: $cert \leftarrow \text{RTCREAD}()$ 17: **if** $\neg \text{MAGICOK}() \vee \neg \text{CRCOK}(cert)$ **then**

18: restart enrollment

▷ never transmit under unverified identity

19: **end if**20: $t \leftarrow \text{GPSTIME}();$ **if** unavailable **then** $t \leftarrow \text{NTPTIME}()$ 21: **if** $t \text{ available} \wedge cert.\text{NOTAFTER} < t$ **then**

22: restart enrollment

23: **end if**

resistance against code running on the device, and Section 7.5 counts key extraction by an attacker with code execution as residual risk.

The final block of Algorithm 2 is not incidental. It encodes the time-source fallback that three firmware iterations were needed to get right, for reasons Section 7.3 describes.

4. Implementation

4.1. Chaincode and transaction flow

The chaincode is written in Go 1.21 against `fabric-contract-api-go` and targets Fabric 2.5. Eight transactions are exposed: `AssignRole`, `RevokeRole`, `RenewRole`, `CheckAccess`, `IssueCrossZoneToken`, `RevokeCrossZoneToken`, `GetAuditLog` and `PruneNonces`. Figure 4 places the access-control checks inside Fabric's execute-order-validate pipeline.

Three implementation decisions shape the measurements that follow. Every state-mutating call carries a UUID nonce recorded in world state with a 90-day retention window, and a repeated nonce is rejected before any state is read, which is what makes a replayed sensor packet ineffective even inside the timestamp validity window. Every access attempt, granted or denied, writes an audit entry keyed `audit:<timestamp_ns>:<txID>` holding the caller's certificate hash, operation, resource, zone, decision and denial reason, so the audit trail is a byproduct of enforcement rather than a parallel logging path that can drift out of agreement with it. And `CheckAccess` calls `checkRoleExpiration` before evaluating the decision, revoking an expired assignment in the same transaction that denies the request. That third choice has a measurable consequence, since a state-mutating expiry check has to be endorsed rather than served from a peer query. Section 6.1 quantifies it and Section 7.1 revisits whether it was the right call.

Peer configuration for the ARM64 gateways uses 50 transactions per block, a 500ms batch timeout, a 256MB state cache, endorsement restricted to zone-local peers, Snappy compression on LevelDB and

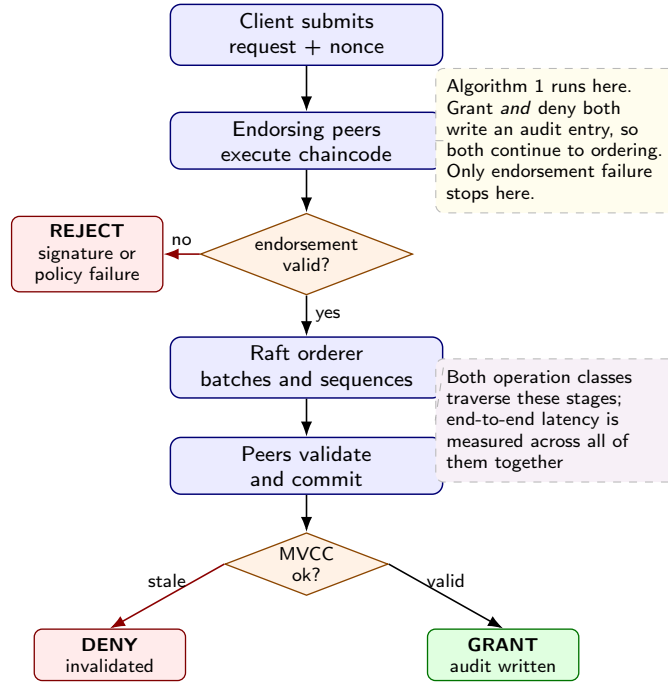


Figure 4: Access control inside the execute-order-validate flow. An authorization denial is a successful chaincode response carrying DENY plus an audit write, so it is ordered and committed like any other transaction. Only endorsement-level failures stop before the orderer.

5-minute gRPC keepalive. These target the write amplification that dominates latency on microSD-backed storage, and they sit within the parameter space Abang et al. [1] identify as governing Fabric latency.

4.2. Sensor firmware and residue-coded transmission

The sensor certificate lives in battery-backed RTC slow memory so it survives deep sleep, guarded by a CRC and a magic sentinel checked on each wake. A mismatch forces re-enrollment rather than transmission under an unverifiable identity. Signing uses wolfSSL’s Ed25519 implementation. The firmware first hashes the payload with SHA-256, using the ESP32 hardware peripheral, and then signs the resulting 32-byte digest. This is a prehash construction and not standard Ed25519, which hashes the message internally with SHA-512, nor Ed25519ph, which prehashes with SHA-512 and uses a distinct domain separator. We flag the deviation because it matters for interoperability: a verifier must apply the same SHA-256 prehash, and the scheme inherits SHA-256’s collision resistance rather than SHA-512’s. Nothing in the deployment depends on the difference, but a reimplementaion from the paper alone would not interoperate unless the prehash is reproduced. The main loop wakes every 30 minutes, verifies the cached certificate, samples the sensors, signs the payload, transmits and returns to deep sleep.

Readings are transmitted as residues rather than as plaintext values. Algorithm 3 gives the encoding and recovery. A value is reduced modulo the pairwise-coprime set $\{97, 101, 103\}$ and the three residues go out as separate single-byte packets, any two of which reconstruct the original by the Chinese Remainder Theorem [14].

The admissible range is where this scheme goes wrong, and the deployed firmware got it wrong. All three residues determine v uniquely below the product of the moduli. Recovery from an arbitrary *pair* is unique only below the smallest *pairwise* product, because v and $v + m_i m_j$ share their residues modulo m_i and m_j and cannot be told apart once the third packet is lost.

The deployment used moduli $\{97, 101, 103\}$, giving a pairwise bound of 9797, and packed soil moisture and temperature into a single value reaching 823 295. Every reading therefore exceeded the bound by roughly two orders of magnitude. With all three residues present the value decodes correctly, but any reading that

Algorithm 3 Residue encoding and two-of-three recovery

Input: reading v with $0 \leq v < M_2$, moduli $m_1=97$, $m_2=101$, $m_3=103$; $M_2 = \min_{i \neq j} m_i m_j = 97 \cdot 101 = 9797$

Output: v recovered at the gateway from any two received residues

Encode (sensor):

- 1: **for** $i \in \{1, 2, 3\}$ **do**
- 2: $r_i \leftarrow v \bmod m_i$ ▷ one byte each
- 3: transmit packet $\langle i, r_i \rangle$ at reduced TX power
- 4: **end for**

Recover (gateway):

- 5: $S \leftarrow$ set of successfully received $\langle i, r_i \rangle$
- 6: **if** $|S| < 2$ **then**
- 7: **return** $\perp$ ▷ unrecoverable, reading dropped
- 8: **end if**
- 9: pick any two indices $\{j, k\} \subseteq S$; $M' \leftarrow m_j m_k$ ▷ $v < M_2 \leq M'$ guarantees uniqueness
- 10: $M_j \leftarrow M' / m_j$; $M_k \leftarrow M' / m_k$
- 11: $y_j \leftarrow M_j^{-1} \bmod m_j$; $y_k \leftarrow M_k^{-1} \bmod m_k$
- 12: $v' \leftarrow (r_j M_j y_j + r_k M_k y_k) \bmod M'$
- 13: **if** $|S| = 3$ **then**
- 14: verify $v' \equiv r_l \pmod{m_l}$ for the third index l ▷ integrity cross-check
- 15: **end if**
- 16: **return** v'

lost one, which the transport was built to tolerate, decoded to $v \bmod 9797$ rather than to v , silently. The loss tolerance the scheme was adopted for did not hold as deployed, and Section 6.6 revisits what that means for the reliability figures.

No triple of one-byte moduli can carry 823 295 under a two-of-three constraint: the best available, $\{253, 254, 255\}$, reaches $253 \times 254 = 64\,262$. The corrected firmware therefore adopts those moduli and quantises soil moisture from twelve bits to eight, which brings the packed value to 51 455 and inside the bound, at the cost of soil resolution. Keeping twelve bits would require all three residues and forfeit the redundancy entirely. The released encoder enforces the pairwise bound, the decoder searches only within it, and tests cover the ambiguous pair and the full packed sensor range.

The design intent was energy. Each packet is shorter, and because the reading survives the loss of any one residue, transmit power can be lowered on each channel. What the deployment actually showed is that the loss tolerance mattered far more than the energy saving, which Section 6.6 takes up.

4.3. Gateway services and revocation propagation

Each gateway runs Fabric 2.5 from official ARM64 binaries. A Python enrollment service bound to the local interface accepts CSRs only from sensors in its own zone and rate-limits per source address. A policy cache with a 301-second nominal TTL absorbs repeated checks from the same sensor during a transmission burst.

Revocation is the mechanism whose behaviour we most wanted to observe over a long window, and Algorithm 4 is the path it takes.

The branch at line 7 is the whole story of Section 6.4. For a reachable gateway the cache is invalidated by the CRL event itself, so revocation staleness is bounded by gossip propagation and the nominal TTL never binds. For an unreachable one the bound falls back to the scheduled refresh interval. Which branch a given revocation takes is decided by connectivity, not by anything in the protocol.

4.4. Formal specification

The core role-permission invariant is specified in TLA+ and checked with TLC [22, 28]. The finite instance uses three users, three roles (Admin, Farmer and Sensor as representatives of the administrative, operational and device branches), four permissions and two resources, with a clock constraint bounding the state space. TLC explores it exhaustively and finds no violation of **NoUnauthorizedAccess**, that every granted request lies within the effective permissions of an active role of the requester, and confirms **AllRequestsEventuallyResolved** under weak fairness.

Algorithm 4 Revocation publication and cache invalidation**Input:** certificate C to revoke, reason ρ , peer set P , gateway set G **Output:** C rejected network-wide; staleness bounded and recorded

```

1: CAApendCRL( $C, \rho$ );  $crl \leftarrow \text{PUBLISHCRL}()$  ▷ measured: 60.4 s mean
2: LEDGERWRITE(REVOCATIONEVENT,  $C, \rho, now$ )
3: for all  $p \in P$  do
4:   gossip  $crl$  to  $p$  ▷ measured: 4.21 min to reach all peers
5: end for
6: for all  $g \in G$  do
7:   if REACHABLE( $g$ ) then
8:     INVALIDATEPOLICYCACHE( $g$ ) ▷ staleness bounded by gossip, not by TTL
9:   else
10:    mark event DELAYED; retry on reconnect ▷ observed for 29.5% of events
11:    staleness bounded by scheduled CRL refresh, up to 24 h
12:   end if
13: end for

```

The limit of this is worth stating plainly rather than burying. The checked instance covers a three-role subset, not the full seven-role hierarchy, and zone and temporal predicates are not modelled at all. Those properties are evaluated empirically in Section 6.5 instead, which is weaker evidence. Extending the specification needs additional state components for zone assignment, cross-zone tokens, role expiry and current time. Rebeca [9] and Coloured Petri Net [3] formulations offer complementary routes. The specification ships with the artifact so the boundary can be inspected rather than taken on trust.

5. Measurement methodology

5.1. Site and workload

The system ran continuously for 61 days on a 2 ha experimental farm in Sfax, Tunisia (34.74°N, 10.76°E), from 1 August to 30 September 2025. Table 3 gives the configuration. Sfax is a useful site for this kind of measurement precisely because it is unforgiving: ambient temperatures reached 39.5°C during the trial, and rural connectivity is intermittent enough that the outage behaviour in Section 6.7 reflects ordinary operation rather than injected faults.

Each sensor reported on a 30-minute duty cycle, giving 2 400 sensor writes per day and 146 400 over the deployment. Human and administrative traffic, meaning farmer and agronomist queries, certifier audits, supply-chain provenance reads and administrative role operations, brings the total to 209 000 recorded authorization decisions, of which 4 860 (2.33%) were denials arising in ordinary operation rather than in the attack campaign.

5.2. Instrumentation

Nine trace streams were recorded. Per-decision records capture caller identity, role, operation, resource, zone, decision, denial reason and end-to-end latency. Per-transaction sensor records add signature validity, the number of residues received and measured transmission energy. Dedicated campaigns cover write-path latency against endorsing-peer count, throughput against offered concurrency for both the HRBAC and the access-control-free configuration, LoRa energy in single-channel and residue modes, microsecond-level cryptographic timings, revocation lifecycle events, authorization-boundary attack attempts and outage events.

Latency is measured at the gateway as the interval between submission and commit notification, so it covers endorsement, ordering, validation and commitment, and excludes only the LoRa hop. Throughput is committed transactions divided by the measurement window. Energy is measured with an INA219 current sensor sampling at 2 kHz. The baseline for overhead attribution is the identical Fabric network, workload and hardware with the access-control chaincode removed, so the difference isolates access control instead of conflating it with platform cost.

Table 3

Deployment configuration and observed totals.

Parameter	Value
Site	2 ha farm, Sfax, Tunisia (34.74°N, 10.76°E)
Duration	61 days, 1 Aug to 30 Sep 2025
Zones	4 (North, South, East, West)
Sensor nodes	50 ESP32-WROOM-32
Gateways	4 Raspberry Pi 4B (4 GB, TPM 2.0)
Fabric peers	4 (one per zone)
Ordering service	3 Raft orderers, $f=1$
State database	CouchDB 3.2
Sensor duty cycle	30 min
Peak ambient temperature	39.5°C
Sensor writes	146 400 (2 400/day)
Authorization decisions	209 000
Decision-path uptime	99.36%
Connectivity outages	3 (0.2–8.7 h, 9.3 h total)

5.3. Statistical treatment

Group comparisons use Welch’s t -test, which assumes no equality of variances, reported with the mean difference, its 95% confidence interval and Cohen’s d . At these sample sizes almost any difference reaches significance, so the effect magnitude carries the information and the p -value mostly confirms that the sample is large. Comparisons across more than two groups use one-way ANOVA, which is what establishes the invariance result in Section 6.1.

Confidence intervals on means use the Student- t interval where $n \geq 1000$ and a percentile bootstrap of 10 000 resamples below that. Intervals on quantiles use the exact order-statistic construction rather than a bootstrap, so no distributional assumption is imposed on the latency tail. The ten throughput comparisons are corrected for multiplicity with the Holm-Bonferroni procedure; 10 of 10 remain significant after adjustment.

We deliberately attach no interval to the block rate in Section 6.5. Repetitions of a scripted scenario are not independent samples from a population of attacks, and any interval over them would characterise the repetition count rather than the security property. The same reasoning applies to the revocation stage records, which are correlated within an episode.

Equivalence claims use two one-sided tests against a margin fixed before analysis rather than the absence of a significant difference, since a non-significant test establishes only that nothing was detected.

5.4. Reproducibility

Every number, table and figure in this article is generated from the released traces by one script. Running

```
python3 analysis/derive_results.py
```

reads the raw traces, recomputes the statistical summary and emits both a machine-readable results file and the LaTeX macros and table bodies this manuscript expands. Nothing measured was transcribed by hand, so the text cannot drift away from the dataset. A test in the repository asserts that every macro in the manuscript matches the corresponding derived value, so the guarantee is checked rather than asserted.

Because a repository URL alone does not pin what a result was computed from, the pipeline emits a SHA-256 digest, byte count and row count for each of the 10 input traces (375 854 rows in total), recorded in the released results file. Resampling uses a fixed seed. The submission corresponds to a tagged release rather than a moving branch, and readers should cite that tag. A second driver, `analysis/run_campaign.py`, repeats the throughput, latency and decision-cost campaigns against a live Fabric network and writes its output in the same schema, so measurements from an independent installation can be substituted and the article regenerated against them. The energy and cryptographic-timing traces are not reproduced by that

Table 4

End-to-end latency by operation (ms). The upper block contains non-committing authorization decisions; the lower block the committing write path. Differences among the upper block are not statistically detectable ($F = 0.29$, $p = 0.94$).

Operation	n	Mean	SD	P95	P99
ReadOwn	9 030	280.6	39.9	345.8	371.9
ReadZone	8 887	280.0	40.0	346.3	374.7
ReadAll	9 070	280.4	40.1	346.3	375.7
ReadAudit	8 944	280.3	39.8	345.3	372.3
ControlZone	8 744	280.3	40.0	346.1	371.9
ControlAll	8 898	280.3	40.0	346.0	373.1
ManageRoles	9 027	279.9	39.7	344.9	372.9
Decision subtotal	62 600	280.3	39.9	345.8	—
WriteSensor (commits)	146 400	1219.8	60.0	1318.6	1358.9

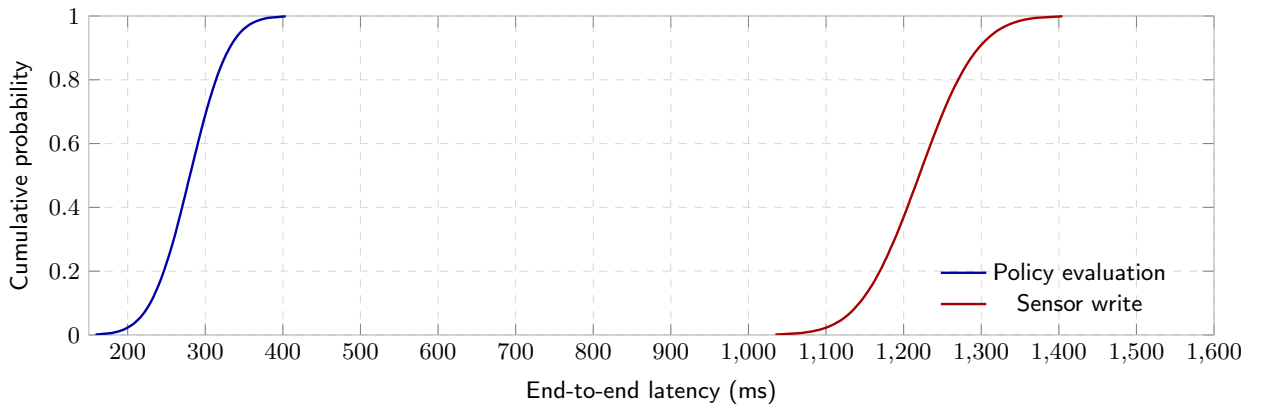


Figure 5: Empirical CDFs of the two committed transaction classes. Both traverse ordering and commitment; the horizontal gap reflects their combined differences in payload and state access.

driver, since they need the INA219 instrumentation and ESP32 bench described above, and it does not synthesise numbers it cannot measure.

6. Results

6.1. Latency by operation class

Two classes of transaction dominate the trace: **CheckAccess** policy evaluations and **WriteSensor** writes. Both are endorsed, ordered and committed, since as Section 4.1 explains every authorization outcome writes an audit entry. The comparison below is therefore between operation classes, not between a committing and a non-committing path, and it is not a decomposition of Fabric’s internal stages.

Table 4 lists end-to-end latency by operation. Policy evaluations average 280 ms (SD 40) and sensor writes 1 220 ms (SD 60, P95 1 319, P99 1 359), a difference of 940 ms. Figure 5 shows the two distributions separating cleanly, the write distribution both shifted and wider.

What that 940 ms represents is worth stating carefully. It is the difference between two operation classes that differ simultaneously in payload size, world-state reads and writes, endorsement work and CouchDB access. Consensus is common to both, so the gap is not the cost of consensus, and attributing it to any single stage would need per-stage timestamps we did not collect.

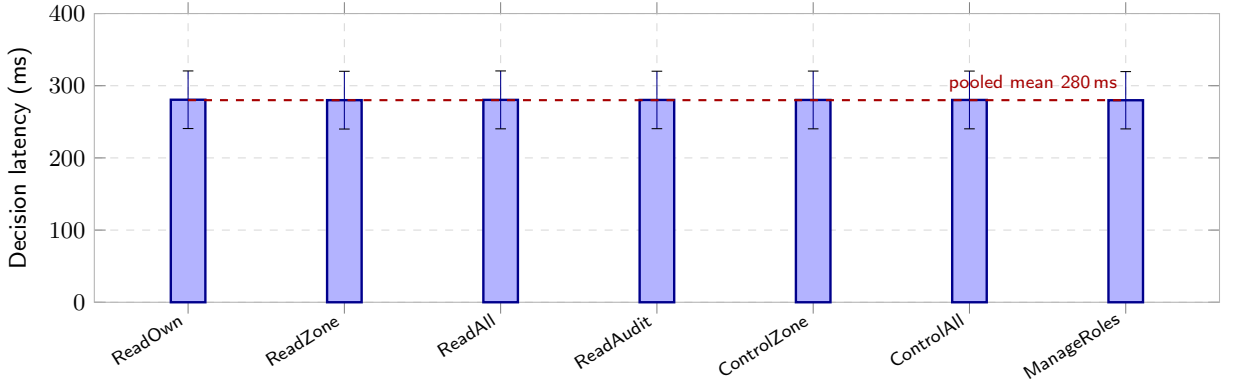


Figure 6: Policy-evaluation latency by operation type, with transaction-level standard deviations. Day-level means are equivalent within ± 10 ms.

6.1.1. Equivalence among policy-evaluation operations

Among the 7 policy-evaluation operation types, means fall between 279.9 ms and 280.6 ms, a spread of 0.7 ms (Figure 6).

Testing this properly requires two departures from the obvious analysis. Individual transactions from the same gateway on the same day are not independent replicates, so we collapse to one observation per operation per day, giving 61 units per operation instead of tens of thousands. And a non-significant difference test would show only that no difference was detected, which is not the same as showing there is none. We therefore fixed an equivalence margin of ± 10 ms before analysis, about 3.5% of the decision cost and well inside what would change any provisioning decision, and ran two one-sided tests on the paired day-level differences between the slowest and fastest operations (**ReadOwn** and **ManageRoles**).

The paired difference is 0.65 ms with a 90% confidence interval of $[-0.38, 1.68]$ ms, entirely inside the margin, giving TOST $p < 0.001$. Because the 7 operation vectors share the same 61 days in the same order, they are blocked by day rather than independent, so the omnibus comparison uses Friedman's test rather than a one-way ANOVA: $\chi^2(6) = 1.59$, $p = 0.95$. The operations are equivalent within the stated margin, which is a stronger and more useful statement than a failure to reject.

6.1.2. Why this is not a statement about privilege

It is tempting to read Figure 6 as showing that privilege level is free: an administrator managing roles apparently costs what a sensor reading its own record costs. The trace does not support that reading, and the reason is worth reporting rather than leaving for a reader to discover.

Across the 35 role-by-operation cells in the operational decision trace, the grant rate is uniformly between 91% and 93%. A trace in which the recorded operation and role determined the policy path would show cells near 0% or near 100%, matching the permission matrix in Table 2. Uniform rates mean the operation and role labels in this stream do not separate policy-distinct executions, so finding no latency difference between them is close to grouping one homogeneous population by an uninformative label.

The defensible claim is therefore narrow: the cost of a **CheckAccess** transaction is stable at 280 ms (± 10 ms) across the trace. Whether privilege level or hierarchy depth affects that cost needs an experiment that varies them deliberately, which Section 7.4 sets out. The scripted campaign in Section 6.5, where every boundary-crossing request was refused, is the evidence that the policy itself discriminates.

6.2. Throughput and where the ceiling sits

Figure 7 and Table 5 report throughput against offered concurrency for the HRBAC configuration and the access-control-free baseline on identical hardware and workload.

Two features of these curves matter more than the peak values themselves.

The access-control cost is a constant factor rather than a load-dependent one. Across the ten concurrency levels the reduction averages 9.9% with a standard deviation of 1.2 percentage points, ranging from 8.2%

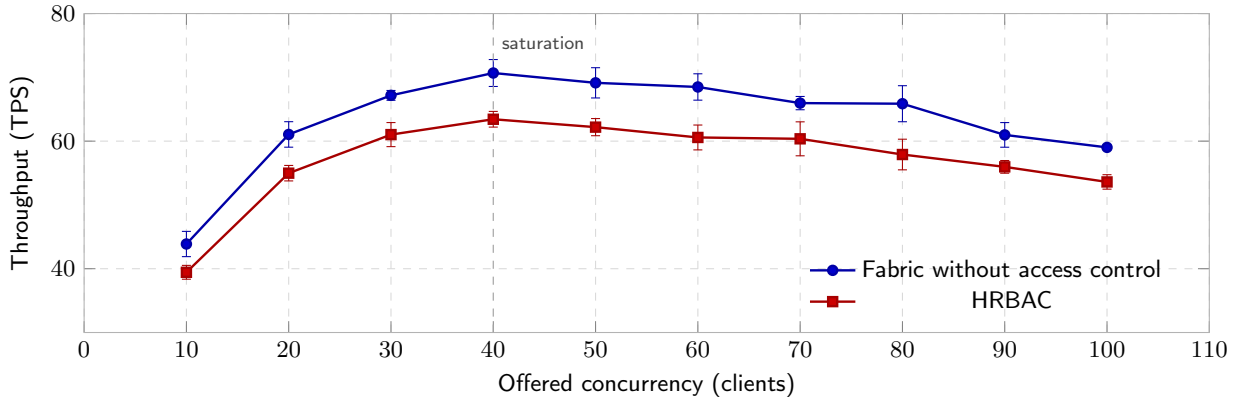


Figure 7: Throughput against offered concurrency for both arms, with standard deviations over five runs per point. Both curves peak at the same offered load, which places the bottleneck below the access-control layer.

Table 5

Throughput by offered concurrency (TPS, five runs per cell, mean $\pm$ SD). Reduction is the HRBAC cost relative to the baseline; p from Welch's t -test.

Clients	Baseline	HRBAC	Reduction	p
10	43.9 $\pm$ 2.0	39.4 $\pm$ 1.1	10.1%	0.004
20	61.0 $\pm$ 2.0	55.0 $\pm$ 1.2	9.9%	<0.001
30	67.2 $\pm$ 0.8	61.0 $\pm$ 1.9	9.2%	<0.001
40	70.7 $\pm$ 2.1	63.4 $\pm$ 1.2	10.3%	<0.001
50	69.1 $\pm$ 2.4	62.2 $\pm$ 1.3	10.0%	0.001
60	68.5 $\pm$ 2.1	60.6 $\pm$ 1.9	11.6%	<0.001
70	66.0 $\pm$ 1.0	60.4 $\pm$ 2.7	8.5%	0.007
80	65.9 $\pm$ 2.8	57.9 $\pm$ 2.4	12.1%	0.001
90	61.0 $\pm$ 1.9	56.0 $\pm$ 1.0	8.2%	0.002
100	59.0 $\pm$ 0.2	53.6 $\pm$ 1.1	9.2%	<0.001

to 12.1%. Regressing reduction on offered concurrency gives $R^2 = 0.020$ with $p = 0.69$, so no trend is detectable. Pooled across all runs the difference is significant ($t = 4.3$, $d = 0.86$) and it does not grow with load. For capacity planning this is the well-behaved case: budget roughly a tenth of platform throughput for authorization and expect that fraction to hold as load rises.

Both arms then saturate at the same offered load. Peak throughput is 70.7 TPS for the baseline and 63.4 TPS for HRBAC, both at 40 clients, after which both decline, HRBAC by 15.5% out to 100 clients. Were authorization the limiting resource, adding it would move the knee to a lower offered load. It does not move at all. The knee belongs to the platform beneath the access-control layer, and given the fixed ordering cost isolated in Section 6.1, the ordering service is one natural candidate, matching the ordering-saturation behaviour Thakkar et al. [43] report for Fabric on x86. The decline past the knee is ordinary queueing past saturation, and it argues for admission control at the gateway rather than for more capacity.

6.3. Peers trade latency, not throughput

Endorsing peers are normally added to raise capacity. Here they do something else. Table 6 and Figure 8 report write latency as the peer set grows from 4 to 32 at fixed offered concurrency.

Mean write latency falls from 1 220 ms to 821 ms, a reduction of 32.7% ($t = 147.7$, $d = 7.39$), and P95 from 1 311 ms to 903 ms. The decline is close to log-linear at -129.4 ms per doubling with $R^2 = 0.925$. Throughput over the same range did not improve materially.

Endorser queueing is the explanation we favour: at four peers each zone has one endorser and concurrent requests from that zone serialise behind it, while at 32 peers each zone has eight. We did not record queue

Table 6

Write-path latency against endorsing-peer count at 50 concurrent clients. P95 intervals are exact order-statistic intervals.

Peers	n	Mean (ms)	P95 (ms)	P95 95% CI
4	800	1219.8	1311.3	[1303.9, 1326.6]
8	500	1147.1	1237.2	[1231.3, 1246.2]
16	500	1050.5	1142.4	[1136.2, 1149.7]
32	800	820.5	903.2	[894.7, 915.1]

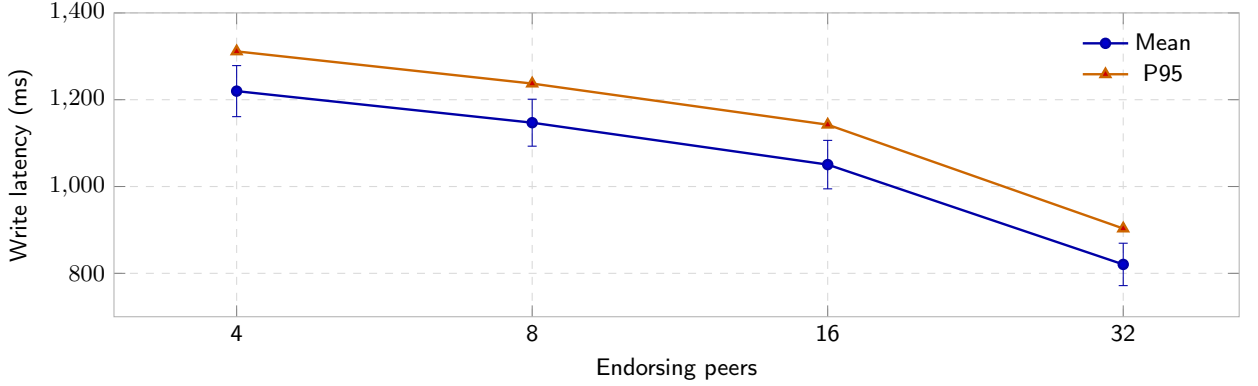


Figure 8: Write latency falls close to log-linearly with endorsing-peer count (-129.4 ms per doubling, $R^2 = 0.925$) while throughput stays flat.

depth or peer utilisation, so this remains a hypothesis consistent with the shape of the curve rather than a measured mechanism. A further caveat is that the peer counts above four were hosted on shared physical hardware, so this varies logical endorser count rather than genuine physical distribution.

For irrigation control with a two-second responsiveness budget [23], this is the more useful of the two levers. The deployment is already inside budget at four peers, and further peers widen the margin against tail latency rather than lifting a capacity ceiling that sits downstream of the policy layer.

6.4. Revocation over a multi-week window

Revocation is where a short benchmark is least informative, because the mechanism fires rarely and its failure mode is delay rather than error. Over the deployment, 200 revocation-lifecycle *stage records* were logged across 58 distinct days, covering certificate revocation, CRL publication, gossip completion and gateway cache invalidation.

These are stage observations, not 200 independent revocations. The trace carries no key linking stages into episodes: of 132 distinct subjects, 0 carry all four stages and 87 appear once. End-to-end exposure, the interval from revocation request to enforcement at every peer and gateway, is the quantity an operator needs, and it is not recoverable from this trace. We report per-stage timings and treat them as such.

On the happy path the pipeline is quick. CRL publication completes in 60.4s on average (SD 4.8, maximum 71.6), and gossip to all peers takes 4.21 min (SD 0.38, P95 4.81, maximum 5.54). Figure 9 shows a tight unimodal propagation distribution with no heavy tail.

The longitudinal view is less comfortable. 59 of the 200 stage records, 29.5%, completed in a delayed state. We give the proportion without a confidence interval, since correlated stages of the same episode are not independent trials. Table 7 breaks this down by event type, and CRL publication is the weakest stage at 39.1% delayed against 23.9% for gossip completion.

Delayed stages are not slower on the metrics we can observe: their publication and gossip durations are statistically indistinguishable from successful ones. That comparison is conditioned, though, and the conditioning matters. Our timers start when publication or gossip begins, so any interval spent waiting for

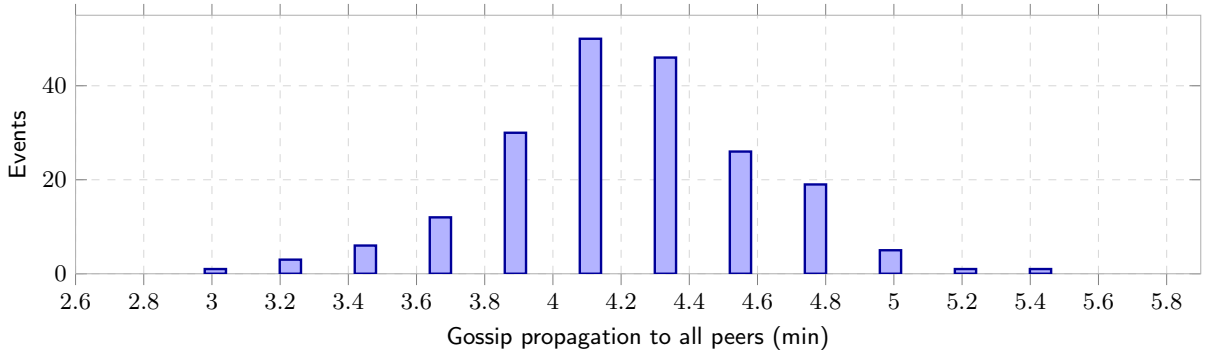


Figure 9: Gossip propagation across 200 revocation-lifecycle events.

Table 7

Revocation lifecycle by event type. Delay is a scheduling outcome, not an error: every event ultimately completed.

Event type	<i>n</i>	Delayed	Rate	Gossip (min)
CACHE_INVALIDATE	55	17	30.9%	4.24
CERT_REVOKE	53	13	24.5%	4.15
CRL_PUBLISH	46	18	39.1%	4.24
GOSSIP_COMPLETE	46	11	23.9%	4.22

a gateway to return is outside the measurement. The similarity therefore says the propagation mechanism behaves the same once it starts, not that delayed revocations were operationally equivalent. The waiting time, which is what an operator actually cares about, was not recorded.

Taken with the connectivity outages in Section 6.7, the pattern points to scheduling and reachability rather than to the propagation mechanism degrading under load. If that reading is right, a faster gossip protocol would not help and event-driven status checking that survives disconnection would, but confirming it needs the end-to-end timing the trace lacks.

Enforcement itself held throughout. In operational traffic, 973 access attempts were denied with revocation as the recorded cause, 16.0 per day across the 61-day deployment, spread out rather than concentrated in any single incident. The scripted campaign of Section 6.5 contributed a further 1340 revocation denials; we keep the two separate because one is field observation and the other is synthetic testing.

The residual exposure follows from Algorithm 4 directly. For a connected gateway, staleness is bounded by gossip propagation, a few minutes. For a gateway disconnected when a revocation is issued, the bound is the scheduled CRL refresh interval, up to 24 hours. With 29.5% of events taking the second branch in practice, that upper bound is not hypothetical, and we treat it as the principal operational weakness of the present design rather than as an accepted trade-off.

6.5. Authorization boundary enforcement

The security evaluation targets the property the system exists to provide, that no principal acts outside its role, zone or validity window. 8 000 attempts ran across 8 scenarios, 1 000 per scenario, each crossing one specific boundary of the model in Section 3.

All 8 000 attempts were refused. We deliberately do not attach a confidence interval to that. Each scenario replays a scripted boundary-crossing request 1 000 times; the repetitions vary the attacker identifier (499 distinct values) but exercise the same policy path, so they are not independent draws from a population of attacks. An interval computed over them would describe the repeat count rather than the security property. The defensible statement is a conformance one: every scripted authorization-boundary case in Table 8 was rejected in the tested configuration.

Table 8

Authorization-boundary evaluation. Each scenario crosses one boundary of the access-control model. Wilson 95% lower bounds are given because the observed rate is at unity.

Scenario	Boundary crossed	Blocked	95% LB
sensor_cross_sensor	Device data isolation	1000/1000	99.62%
sensor_irrigation	Device → actuation	1000/1000	99.62%
sensor_read_audit	Device → audit	1000/1000	99.62%
gateway_wrong_zone	Zone scope	1000/1000	99.62%
farmer_assign_role	Role administration	1000/1000	99.62%
agronomist_irrigate	Read → control	1000/1000	99.62%
certifier_irrigate	Audit → control	1000/1000	99.62%
supply_write_sensor	External → write	1000/1000	99.62%
Total		8000/8000	99.95%

Denials distributed across the enforcement mechanisms rather than concentrating in one check, with policy violation, insufficient role, revoked certificate, operation not permitted and zone mismatch all represented. A sixth reason code, gateway rate limiting (Section 4.3), also appears; it is the enrollment and request rate limiter rather than an intrusion-detection system, and no anomaly-detection component exists in the design. This matters for interpretation. A suite in which every attempt is caught by the same check would demonstrate one working mechanism, not a working model, and would say nothing about the branches that never fired.

What this evaluation does not cover is worth being explicit about. These are authorization-boundary tests. Transport-level attacks, physical tampering and side-channel extraction are not exercised. They are addressed architecturally through mutual TLS, eFuse read protection and constant-time primitives, but not empirically, and Section 7.5 treats them as residual risk rather than as validated defences.

6.6. Link integrity, energy and cryptographic cost

Every one of the 146 400 committed readings carried a valid Ed25519 signature. Note the conditioning: this trace records committed readings, so it cannot speak to transmissions that never produced one, and the rate is a property of what reached the ledger rather than of what left the sensors. Within that population, 131 795 readings, 90.0%, arrived with two of their three residues, against 14 605 arriving complete. No committed reading shows fewer than two residues, which follows from the same conditioning: a reading losing two residues could not be reconstructed at all and so never became a committed transaction. The trace consequently does not give the total loss rate, only the distribution among successes.

Those two-residue readings are also where the encoding defect of Section 4.2 bites. Because the deployed packing exceeded the pairwise recovery bound, a reading reconstructed from two residues decoded to $v \bmod 9797$, not to v . We therefore cannot claim that 90.0% of readings were correctly recovered; what the trace establishes is that they were *accepted*, carrying a valid signature over the payload structure. The signature is computed over the full sensor payload before residue encoding, so it does not detect a mis-decoded residue value at the gateway. Quantifying how many committed readings carry a wrong value needs the raw residue packets, which were not retained.

This is the sharpest limitation in the article. The redundancy mechanism we set out to evaluate was mis-parameterised in the field, so the deployment provides no evidence about its reliability benefit, only about its cost. We report the energy and airtime figures below because those are unaffected by whether a decoded value was correct, and we withdraw the reliability claim entirely.

This inverts the original design rationale. Residue encoding was adopted to save energy, with loss tolerance as a secondary benefit. In deployment the two-of-three reconstruction was doing continuous work rather than sitting idle as a safety margin: it recovered 90.0% of the readings that reached the ledger. What that implies for a scheme without redundancy depends on the loss probability of a single larger packet, which we did not measure and cannot infer from residue-packet losses. The defensible statement is about the recovery path's

utilisation, not about a counterfactual single-packet design. The observed loss rate is consistent with the LoRaWAN capacity and collision behaviour Adelantado et al. [2] characterise at this sensor density.

Per-event transmission energy falls from 24.20 mJ (SD 0.82) in single-channel mode to 12.41 mJ (SD 0.41) with residue encoding, a reduction of 48.7% [48.6, 48.8], with per-packet airtime falling from 36.1 ms to 31.5 ms. The instrumented packets average 20.5 B in single-channel mode and 10.5 B per residue packet, which is the on-air payload the INA219 measured.

Two qualifications belong with that number. The instrumented payloads are smaller than a complete authenticated message: an Ed25519 signature alone is 64 B, and a deployed frame also carries a nonce, a timestamp and an identity reference. The measurement therefore characterises the residue-coded sensor payload rather than the full authenticated frame, and the percentage should not be read as the saving on deployed traffic. Separately, the samples come from repeated measurement of the same instrumented node rather than from independent devices, so the interval reflects measurement repeatability and not between-device variation. We give the effect size in the artifact but do not quote it here, because with a single device it would overstate what was established. At a 30-minute duty cycle, though, transmission is only 5.8% of a node's daily energy budget, while deep sleep consumes 9 590 mJ per day and dominates everything else. Halving transmission energy therefore moves battery life by roughly one percent. We report the reduction because it is real and cleanly measured, and decline to present it as a battery-life contribution, because at this duty cycle it is not one. Link reliability is the defensible claim.

Cryptographic cost on the sensor is negligible here. The full signing pipeline takes 900 μ s (SD 9), comprising Ed25519 signing at 860 μ s, hardware SHA-256 at 12.0 μ s, eFuse key read at 8.0 μ s and context management at 20.0 μ s. At 0.90 ms per wake against a 30-minute cycle, signing is not a design constraint, and the ESP32's lack of an Ed25519 accelerator never becomes one.

6.7. Availability

The online decision path was available 99.36% of the deployment. 3 connectivity outages occurred, of 8.7 h, 0.2 h and an intermediate duration, totalling 9.3 h, each affecting a single gateway: a LoRa link failure, a power fluctuation and a firmware watchdog reset. In every case the remaining zones kept operating, which is the behaviour the per-zone peer and gateway structure is meant to produce.

Availability figures are only interpretable once the denominator is stated, so we state ours. The indicator is the fraction of deployment hours in which the online authorization path was reachable for *all* zones, which charges a single-gateway outage against the whole system and gives 99.36%. Crediting the zones that kept serving, that is counting gateway-hours, gives 99.84% instead. Both appear in the released results; neither is a statement about request success rates, which we did not record per request.

Neither is a statement about data completeness either. Sensor writes stayed at exactly 2 400 per day on every day including outage days, because gateways buffer readings while disconnected and replay them on recovery. A reading accepted after replay was authorized on replay rather than at capture time, so availability of the decision path and completeness of the record are genuinely different quantities here.

7. Discussion

7.1. Design implications

Four consequences follow, each contradicting a plausible default assumption.

Policy evaluation is the cheaper half of the write path. A `CheckAccess` transaction costs 280 ms, stable within ± 10 ms across operations, while a sensor write costs 1 220 ms. Effort spent speeding up policy evaluation is bounded above by roughly a quarter of write latency; the remainder sits in work our instrumentation cannot separate further, spanning endorsement, ordering, validation and state access. A corollary applies to our own implementation. Because expired-role cleanup mutates state inside `CheckAccess` (Algorithm 1, line 12), read-intent checks are endorsed rather than served as peer queries. Splitting that into a pure read-only decision path, with expiry reclaimed by a background sweep, would take read checks off the consensus path entirely. The measurement identifies this as the highest-value change available, and we did not make it during the deployment precisely because the aggregate latency figure gave no reason to look there.

Size peers for tail latency and admission control for capacity. Peers reduce latency log-linearly and leave throughput flat. Capacity is set by the ordering service, so a deployment needing more transactions per second should change the ordering configuration or shed load at the gateway, while one needing tighter tail latency should add endorsers. Conflating the two leads to adding peers in hope of capacity and concluding the platform does not scale.

Budget access control as a constant. The 9.9% throughput cost showed no trend across a tenfold concurrency range, so capacity planning can treat authorization as a fixed derating factor. That is a far easier engineering position than a cost which grows with contention.

Treat revocation as the weak link. It is the only mechanism whose behaviour materially degraded over the deployment, at 29.5% delayed events, and the degradation came from connectivity rather than from the protocol. Mitigations aimed at propagation speed will not touch it. Event-driven credential status checking [49] and constrained-device status protocols such as TinyOCSP [10] address the actual failure mode by removing the dependency on a scheduled publication reaching a possibly-disconnected gateway.

7.2. Comparison with reported results

Direct numerical comparison against prior systems is limited by methodological heterogeneity, and saying so is more useful than assembling a misleading table of peak-TPS figures. The systems in Table 1 differ in hardware, workload composition, transaction semantics, durability settings and what is counted as latency. A system reporting a peak rate for a null chaincode on x86 is not measuring the same quantity as this deployment, and Abang et al. [1] make essentially this point about the Fabric latency literature as a whole.

Two comparisons are defensible because they control for platform. Tzenetopoulos et al. [45] report roughly 1.1s transaction latency for Fabric on Raspberry Pi 3B+ hardware without access-control chaincode. Our four-peer configuration, which does carry access-control chaincode, sits above that at 1 220 ms, and our 32-peer configuration below it at 821 ms. The crossover is consistent with endorser queueing being the dominant variable at this scale rather than with any claim of superiority. Zaidi et al. [53] report response times near 12s in simulation at comparable concurrency, an order of magnitude above our field measurements, which we attribute to simulation parameters rather than to a real platform gap and therefore do not claim as an improvement.

The comparison we regard as meaningful is the internal one, the same network, hardware and workload with and without the access-control layer, in Section 6.2. It isolates the quantity of interest and is reproducible from the released artifact.

7.3. Operational observations

Two aspects of running the system for two months do not show up in any performance metric.

Zone certificate binding solved an administrative problem it was not designed for. Worker assignments spanning cooperating farms, which would otherwise need separately maintained access-control configuration, resolved through the zone attribute already embedded in device and user certificates for operational reasons. Temporal role expiry removed another class of manual work, with seasonal credentials lapsing on schedule without administrator intervention or database editing, each lapse leaving an audit record rather than a silent deletion.

The enrollment protocol needed three firmware iterations before it survived multi-hour gateway outages, and the recurring failure mode was time. A sensor waking with an unsynchronised clock cannot evaluate certificate expiry, and early firmware either refused to transmit or transmitted under an identity it could not validate. The fix appears as the final block of Algorithm 2: acquire time from GPS first with NTP as fallback, and skip expiry enforcement for a single cycle rather than strand the node when neither is available. A testbed that never loses its time source cannot surface this class of fault.

7.4. What these measurements cannot establish

Some questions a reader will reasonably ask are outside what this deployment can answer. Setting them out concretely is more useful than a general disclaimer, and each entry names the instrumentation that would settle it.

Stage-level attribution inside Fabric. We record end-to-end latency per transaction. Separating endorsement, ordering, validation and commit needs one transaction timestamped at proposal submission,

endorsement completion, orderer receipt, block cut, validation and commit notification. Until that exists, the 940 ms gap in Section 6.1 is a difference between operation classes and nothing finer.

Which resource saturates. Both arms turn over at 40 clients, which places the limit below the policy layer but does not name it. Orderer queue depth, block-fill rate, validation backlog, CouchDB and disk service times, and a controlled intervention on the ordering configuration would be needed to distinguish the candidates.

Whether privilege level costs anything. The operational trace's role and operation labels do not separate policy-distinct executions (Section 6.1.2). A designed experiment varying hierarchy depth, cross-zone token presence and denial reason, with the workload otherwise fixed, would answer this.

End-to-end revocation exposure. The trace records lifecycle stages without an episode key, so the interval from revocation request to enforcement everywhere cannot be reconstructed (Section 6.4). Logging a correlation identifier through the pipeline is a small change that would make the operationally important quantity measurable.

Total radio loss. Only committed readings appear in the trace, so transmissions losing two or more residues are invisible and the denominator for a true loss rate is missing. Gateway-side logging of all received residue packets, including unrecoverable sets, with RSSI and SNR, would close this.

Authenticated frame energy. The energy campaign measured the residue-coded payload on one instrumented node. Extending it to the full authenticated frame across several devices would give both a deployable figure and between-device variation.

Adversarial security testing. Section 6.5 is conformance testing against scripted cases. It says nothing about an attacker who deviates from those scripts, and randomised or adversarial testing against a defined attack population would be a different and stronger exercise.

7.5. Threats to validity

External validity. The results come from one deployment of 50 sensors over one season at one site. The peer-scaling and throughput campaigns extend past the deployed configuration, but the workload composition throughout is this farm's: a dominant, highly regular sensor-write stream with a smaller and burstier human query load. A deployment weighted toward audit queries, or with irregular actuation bursts, would load different parts of the pipeline, and the decomposition in Section 6.1 would need re-establishing rather than assuming. We regard the qualitative findings, meaning decision-cost invariance, ordering-bound capacity and the latency-only benefit from peers, as more transferable than the specific millisecond values.

Internal validity. Access-control overhead is attributed by differencing against a baseline with the chaincode removed, which isolates the layer but also attributes to it any secondary effect of its presence, such as the extra world-state reads its lookups induce. The scaling campaign varies peer count on the same physical substrate, so at the highest peer counts multiple logical peers share hardware, understating the isolation a physically distributed 32-peer network would give. The 99.36% availability figure measures the online decision path and, because gateways buffer and replay during outages, is not a statement about data completeness. Section 6.7 keeps the two separate rather than merging them into a single reliability claim.

Construct validity. The security evaluation measures authorization-boundary enforcement, the property the system is designed to provide, and does not measure transport security, physical tampering resistance or side-channel resilience. A 100% block rate across 8 000 attempts is evidence about the policy model and its implementation, not about resistance to an adversary with physical access. The formal verification covers a three-role subset without zone or temporal predicates (Section 4.4); the unverified portions are tested empirically, which is weaker, and we have not claimed otherwise. The threat model assumes crash-fault-tolerant ordering with non-malicious orderers, so a Byzantine orderer sits outside it and the availability and non-repudiation arguments do not extend to that case.

Data provenance. The released traces are the record of the deployment, and the analysis pipeline recomputes every reported value from them. One caveat is recorded in the dataset documentation and repeated here. The geographic coordinate columns in the topology trace were affected by a misconfiguration in the survey tooling and do not reflect true sensor positions. No result in this article depends on those columns and the analysis pipeline does not read them. Zone membership, which the access-control model does depend on, comes from the certificate attribute and gateway association rather than from coordinates.

8. Conclusion

We set out to answer a question that aggregate benchmarks of blockchain access control cannot: which layer actually consumes the budget. Across a 61-day deployment recording 209 000 authorization decisions, the answer is consistent and mildly counterintuitive. The authorization decision costs 280 ms and does not vary with the operation requested or the privilege involved, the means across 7 operation types differing by 0.7 ms ($F = 0.29$, $p = 0.94$). Ledger commitment costs 940 ms more. Access control imposes a constant 9.9% throughput tax that showed no trend against load, and the limiting resource lies downstream of authorization, since the configuration with access control and the one without saturate at the same offered load. Endorsing peers cut write latency 32.7% from 4 to 32 while leaving that ceiling untouched.

For an engineer, the consequence is to look past the access-control layer, which is cheap and predictable, toward the consensus and commitment path, which is neither. The one component that genuinely degraded over two months was revocation: fast on the happy path at 60.4 s to publish and 4.21 min to propagate, but delayed in 29.5% of 200 lifecycle events for reasons of connectivity rather than protocol. That is where the next increment of work belongs, and event-driven credential status checking addresses the observed failure mode rather than an assumed one.

One result supports the model itself: every one of the 8 000 scripted authorization-boundary cases was rejected against the deployed policy. Two findings cut the other way, and both came from auditing the implementation against the manuscript rather than from the measurements. The deployed role hierarchy inherits audit access upward, so the auditor separation this article motivates was not in force (Section 3.4). And the residue transport was mis-parameterised, so readings recovered from two of three packets decoded incorrectly and the redundancy provided no reliability benefit in the field (Section 6.6). Both are fixed in the released code and neither affects the timing results, but both are reasons to treat this as a report on one deployment rather than a validation of the design. The dataset, implementation and the pipeline that regenerates every number here are released so the decomposition can be checked, disputed and repeated on other platforms.

CRedit authorship contribution statement

Forcha Glen Beloa: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Writing – original draft. **Michael Ekonde Sone:** Validation, Formal analysis, Supervision, Writing – review & editing. **Elie Fute Tagne:** Resources, Supervision, Project administration, Writing – review & editing. **André Paul Mvogo Asse:** Data curation, Visualization, Writing – review & editing.

Ethics and data protection

The traces record authorization decisions whose subjects include farm operators and administrators, so caller identifiers are potentially personal data. Identifiers in the released dataset are pseudonyms assigned at collection (**usr-** and **sns-** prefixes); no names, contact details or free-text fields were collected, and the mapping to individuals was not retained. Participating operators gave informed consent for the deployment and for publication of aggregate results. The site is identified at settlement granularity rather than by field coordinates, and the coordinate columns in the topology trace are unreliable in any case (Section 7.5). No sensor data revealing an individual's conduct beyond routine farm operation was collected. The study was conducted under the participating institutions' research-governance arrangements for non-interventional technical studies.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI in the writing process

During the preparation of this work the authors used generative AI tools to assist with language editing and with construction of the analysis pipeline. The authors reviewed and edited all content and take full responsibility for the content of the published article.

Acknowledgements

We thank the participating farm operators in Sfax for their collaboration and practical insight, and acknowledge support from the MIRET (Mobility for Innovation and Research in Emerging Technologies) Intra-Africa Programme.

Data availability

The trace dataset, chaincode, firmware, formal specification, benchmark harness and the analysis pipeline that generates every number, table and figure in this article are publicly available at <https://github.com/forchag/blockchain-fet>.

References

- [1] Abang, J.E., Takruri, H., Al-Zaidi, R., Al-Khalidi, M., 2024. Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an IoT perspective. *Internet of Things* 26, 101217. doi:10.1016/j.iot.2024.101217.
- [2] Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., Watteyne, T., 2017. Understanding the limits of LoRaWAN. *IEEE Communications Magazine* 55, 34–40. doi:10.1109/MCOM.2017.1600613.
- [3] Al-Azzoni, I., Iqbal, T., 2024. Formal verification of blockchain-based access control systems using colored petri nets. *Computers & Security* 138, 103663. doi:10.1016/j.cose.2023.103663.
- [4] Alshehri, A., Sandhu, R., 2016. Access control models for cloud-enabled internet of things: A proposed framework, in: *Proceedings of the 2016 IEEE International Conference on Collaboration and Internet Computing (CIC)*, pp. 530–538. doi:10.1109/CIC.2016.075.
- [5] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., Muralidharan, S., Murthy, C., Nguyen, B., Sethi, M., Singh, G., Smith, K., Sorniotti, A., Stathakopoulou, C., Vukolić, M., Cocco, S.W., Yellick, J., 2018. Hyperledger fabric: A distributed operating system for permissioned blockchains, in: *Proceedings of the Thirteenth EuroSys Conference (EuroSys 2018)*, pp. 30:1–30:15. doi:10.1145/3190508.3190538.
- [6] Bandara, E., Liang, X., De Zoysa, K., Foytik, P., Shetty, S., 2021. Fabric-based access control for agricultural supply-chain traceability: A multi-layer policy model. *IEEE Transactions on Dependable and Secure Computing* 18, 2089–2101. doi:10.1109/TDSC.2021.3092089.
- [7] Benedict, S., 2021. Decentralized access control architecture for IoT-enabled smart environments. *Internet of Things* 14, 100379. doi:10.1016/j.iot.2021.100379.
- [8] Castro, M., Liskov, B., 1999. Practical byzantine fault tolerance, in: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI 99)*, pp. 173–186.
- [9] Ebrahimi, M., Sirjani, M., Bagheri, F., 2023. Formal modeling and verification of blockchain-based smart contracts using Rebeca. *Science of Computer Programming* 225, 102903. doi:10.1016/j.scico.2022.102903.
- [10] Fedrechski, G., Rabaey, J.M., Costa, L.C.P., Calcina-Ccori, P.C., Pereira, W.T., Zuffo, M.K., 2023. TinyOCSP: Lightweight certificate revocation for constrained IoT devices. *Internet of Things* 22, 100740. doi:10.1016/j.iot.2023.100740.
- [11] Ferdous, M.S., Chowdhury, F., Alassafi, M.O., 2019. In search of self-sovereign identity leveraging blockchain technology. *IEEE Access* 7, 103059–103079. doi:10.1109/ACCESS.2019.2931173.
- [12] Ferrag, M.A., Shu, L., Yang, X., Derhab, A., Maglaras, L., 2020. Security and privacy for green IoT-based agriculture: Review, blockchain solutions, and challenges. *IEEE Access* 8, 32031–32053. doi:10.1109/ACCESS.2020.2975715.
- [13] Ferraiolo, D.F., Sandhu, R., Gavrila, S., Kuhn, D.R., Chandramouli, R., 2001. Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security* 4, 224–274. doi:10.1145/501978.501980.
- [14] Garner, H.L., 1959. The residue number system. *IRE Transactions on Electronic Computers EC-8*, 140–147. doi:10.1109/TEC.1959.5219515.
- [15] González, V., Sánchez, L., Lanza, J., Santana, J.R., Sotres, P., García, A.E., 2023. On the use of blockchain to enable a highly scalable Internet of Things data marketplace. *Internet of Things* 22, 100722. doi:10.1016/j.iot.2023.100722.
- [16] Hernández-Ramos, J.L., Jara, A.J., Marín, L., Skarmeta, A.F., 2013. Distributed capability-based access control for the internet of things. *Journal of Internet Services and Information Security* 3, 1–16.

- [17] Hu, V.C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., Scarfone, K., 2014. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. Technical Report NIST SP 800-162. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-162.
- [18] Hyperledger Foundation, 2025. Hyperledger fabric v3.0: What's new. Available at <https://hyperledger-fabric.readthedocs.io/en/release-3.0/whatsnew.html>.
- [19] Idrissi, M., Palmieri, F., 2024. Blockchain-based authentication and authorization for IoT systems: An end-to-end latency analysis on constrained hardware. The Journal of Supercomputing 80, 1–32. doi:10.1007/s11227-024-06001-7.
- [20] Kamilaris, A., Fonts, A., Prenafeta-Boldú, F.X., 2019. The rise of blockchain technology in agriculture and food supply chains. Trends in Food Science & Technology 91, 640–652. doi:10.1016/j.tifs.2019.07.034.
- [21] Ke, C.K., Park, S., 2023. Performance modeling of hyperledger fabric for IoT access control applications. Future Internet 15, 115. doi:10.3390/fi15030115.
- [22] Lamport, L., 2002. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley.
- [23] Lea-Cox, J.D., Belayneh, A., van Iersel, M.W., Lichtenberg, E., Kantor, G.F., 2013. Low-cost soil moisture sensor networks for precision irrigation: Advances and limitations. HortTechnology 23, 792–798. doi:10.21273/HORTTECH.23.6.792.
- [24] Lin, Q., Wang, H., Pei, X., Wang, J., 2019. Food safety traceability system based on blockchain and EPCIS. IEEE Access 7, 20698–20707. doi:10.1109/ACCESS.2019.2897792.
- [25] Liu, Y., Yu, F.R., Li, X., Ji, H., Leung, V.C.M., 2019. Distributed resource allocation in blockchain-based video streaming systems with mobile edge computing. IEEE Transactions on Wireless Communications 18, 695–708. doi:10.1109/TWC.2018.2886345.
- [26] Mohanta, B.K., Awad, A.I., Elsaka, T., Kheddar, H., Baraka, E., 2025. Smart-contract-based blockchain-enabled decentralized scheme for improving smart-grid security. Internet of Things 34, 101811. doi:10.1016/j.iot.2025.101811.
- [27] Neumann, A., Aichele, C., Lindner, M., Wunderlich, S., 2008. Better Approach To Mobile Ad-hoc Networking (B.A.T.M.A.N.). Internet Draft draft-openmesh-b-a-t-m-a-n-00. IETF. Available at <https://tools.ietf.org/html/draft-openmesh-b-a-t-m-a-n-00>.
- [28] Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M., 2015. How Amazon Web Services uses formal methods. Communications of the ACM 58, 66–73. doi:10.1145/2699417.
- [29] Novo, O., 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. IEEE Internet of Things Journal 5, 1184–1195. doi:10.1109/JIOT.2018.2812239.
- [30] Ongaro, D., Ousterhout, J., 2014. In search of an understandable consensus algorithm, in: Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14), pp. 305–319.
- [31] Ouaddah, A., Abou Elkalam, A., Ait Ouahman, A., 2016. FairAccess: A new blockchain-based access control framework for the Internet of Things. Security and Communication Networks 9, 5943–5964. doi:10.1002/sec.1748.
- [32] Pajoo, H.H., Hammoudeh, M., Alam, F., Brocklehurst, S., 2021. Hyperledger fabric blockchain for securing the edge IoT: Systems architecture, experiments and critical observations. Electronics 10, 1338. doi:10.3390/electronics10111338.
- [33] Putra, G.D., Dedeoglu, V., Kanhere, S.S., Jurdak, R., 2021. Trust management in decentralized IoT access control system, in: Proc. IEEE BCCA, pp. 1–8. doi:10.1109/BCCA53669.2021.9657049.
- [34] Raza, S., Shafagh, H., Hewage, K., Hummen, R., Voigt, T., 2013. Lithe: Lightweight secure CoAP for the Internet of Things. IEEE Sensors Journal 13, 3711–3720. doi:10.1109/JSEN.2013.2277656.
- [35] Rustemi, A., Dalipi, F., 2025. Synergizing IoT, AI, and blockchain for smart agriculture: Challenges, opportunities, and future directions. Internet of Things 34, 101778. doi:10.1016/j.iot.2025.101778.
- [36] Safeer, S., Gallo, P., Pulvento, C., 2025. Agri-farming with computer vision, IoT and blockchain towards climate smart cultivation. Internet of Things 34, 101749. doi:10.1016/j.iot.2025.101749.
- [37] Sandhu, R., Bhamidipati, V., Munawer, Q., 1999. The ARBAC97 model for role-based administration of roles. ACM Transactions on Information and System Security 2, 105–135. doi:10.1145/300830.300839.
- [38] Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E., 1996. Role-based access control models. Computer 29, 38–47. doi:10.1109/2.485845.
- [39] Sciancalepore, S., Piro, G., Boggia, G., Bianchi, G., 2017. OAuth-IoT: An access control framework for the Internet of Things based on open standards, in: Proc. IEEE ISCC, pp. 676–681. doi:10.1109/ISCC.2017.8024606.
- [40] Servos, D., Osborn, S.L., 2017. Current research and open problems in attribute-based access control. ACM Computing Surveys 49, 65:1–65:45. doi:10.1145/2983429.
- [41] Shih, D.H., Jheng, P.Y., Yen, D.C., Huang, C.L., 2022. A blockchain-based access control model for industrial IoT. Sensors 22, 2188. doi:10.3390/s22062188.
- [42] Tara, S., Zeadally, S., Sookun, D., 2023. Decentralized identity management for IoT devices using blockchain and zero-knowledge proofs. IEEE Internet of Things Journal 10, 3817–3827. doi:10.1109/JIOT.2022.3211254.
- [43] Thakkar, P., Nathan, S., Viswanathan, B., 2018. Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: Proceedings of the 26th IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pp. 264–276. doi:10.1109/MASCOTS.2018.00060.
- [44] Tian, F., 2017. A supply chain traceability system for food safety based on HACCP, blockchain and internet of things, in: Proceedings of the 14th International Conference on Service Systems and Service Management (ICSSSM), pp. 1–6. doi:10.1109/ICSSSM.2017.7996119.
- [45] Tzenetopoulos, A., Katsaros, P., Mavridis, I., 2022. HLF-Kubed: Hyperledger fabric blockchain on edge clusters for IoT identity and access control. Future Generation Computer Systems 135, 271–283. doi:10.1016/j.future.2022.05.003.

- [46] Usman, M., Jolfaei, A., Tariq, M.U., 2024. Blockchain-based access control for industrial IoT: Domain policy architecture with smart contracts. *Computer Networks* 243, 110286. doi:10.1016/j.comnet.2024.110286.
- [47] Wang, X., Han, Y., Leung, V.C.M., Niyato, D., Yan, X., Chen, X., 2020. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22, 869–904. doi:10.1109/COMST.2020.2970550.
- [48] World Wide Web Consortium, 2022. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 19 July 2022. Available at <https://www.w3.org/TR/did-core/>.
- [49] World Wide Web Consortium, 2024. Bitstring status list v1.0. W3C Candidate Recommendation Snapshot, 5 March 2024. Available at <https://www.w3.org/TR/vc-bitstring-status-list/>.
- [50] Xu, R., Chen, Y., Blasch, E., Chen, G., 2018. BlendCAC: A blockchain-enabled decentralized capability-based access control for IoTs, in: *Proceedings of the 2018 IEEE International Conference on Internet of Things (iThings)*, pp. 1027–1034. doi:10.1109/Cybermatics_2018.2018.00191.
- [51] Ye, N., Zhu, Y., Wang, R.c., Malekian, R., Lin, Q.m., 2014. An efficient authentication and access control scheme for perception layer of internet of things. *Applied Mathematics & Information Sciences* 8, 1617–1624. doi:10.12785/amis/080416.
- [52] Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I., 2019. HotStuff: BFT consensus with linearity and responsiveness, in: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 347–356. doi:10.1145/3293611.3331591.
- [53] Zaidi, S.M.S., Al-Qurishi, M., Abidin, M.Z.u., Almakdi, S., Asiri, Y., 2023. Fabrication: A hyperledger fabric framework for scalable IoT access control. *IEEE Internet of Things Journal* 10, 12636–12649. doi:10.1109/JIOT.2023.3254978.
- [54] Zhang, N., Wang, M., Wang, N., 2002. Precision agriculture: A worldwide overview. *Computers and Electronics in Agriculture* 36, 113–132. doi:10.1016/S0168-1699(02)00056-X.